

Lesson 1 — The Loop Without Pixels

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

A game is three things

1. **State** — the numbers it remembers
2. **A loop** — which changes them
3. **Output** — which shows them

Only the third needs graphics.

The loop

```
while playing:
    show_room()           # RENDER
    command = input("> ")  # INPUT
    do_command(command)   # UPDATE
```

A text game **waits** at input. A graphical game cannot. That is the only real difference.

State

```
player_health = 20
current_room = "cavern"
inventory = []
playing = True
```

Write these on paper, retype them tomorrow, and the game carries on. That is why saving is possible.

The world as data

```
ROOMS = {
    "cavern": {
        "description": "A dark cavern.",
        "exits": {"north": "tunnel"},
    },
}
```

```
def go(direction):
    global current_room
    exits = ROOMS[current_room]["exits"]
    if direction in exits:
        current_room = exits[direction]
    else:
        print("You cannot go that way.")
```

One function, any number of rooms.

global — the trap

```
def cmd_quit(rest):
    global playing      # WITHOUT this...
    playing = False     # ...Python makes a
    NEW local           # variable, changes
                        # it, throws
                        # it away, and
    reports NO ERROR.
```

Being forgiving about input

```
words = input("> ").strip().lower().split()
if not words:
    continue
verb = words[0]
rest = words[1:]
```

Clamping

```
health = max(0, health - damage) # never
below 0
health = min(MAX, health + heal) # never
above MAX
```

Words

State — what the game remembers. **Dictionary** — `{"key": value}`. **Data-driven** — behaviour from data, not code.

Section A — Recall

A1. What are the three jobs in a game loop, in order? Give the Python that does each one in a text game. [3 marks]

A2. What does *state* mean, and what does it have to do with being able to save a game? [2 marks]

A3. What is the one real difference between a text game's loop and a graphical game's loop? [3 marks]

A4. What does `max(0, health - damage)` do, and why would a player care? [2 marks]

A5. Give two advantages of storing rooms in a dictionary rather than in a chain of `if` statements. [3 marks]

Section B — Predict the output

Write your answer before running anything.

B1. What does this print? [3 marks]

```
health = 20
health = health - 5
health = health - 30
print(max(0, health))
```

B2. What happens when you type `quit`? Be precise — there is no error message.

[4 marks]

```
playing = True

def cmd_quit():
    playing = False

while playing:
    if input("> ") == "quit":
        cmd_quit()
```

B3. The player is in `"cavern"` and types `go north`. What is printed, and what is `current_room` afterwards?

[3 marks]

```
ROOMS = {
    "cavern": {"exits": {"north": "tunnel"}},
    "tunnel": {"exits": {"south": "cavern"}},
}
```

B4. The player types `GO NORTH` (capitals). What happens with each version, and why does it matter which you use?

[4 marks]

```
# version A
words = input("> ").split()

# version B
words = input("> ").strip().lower().split()
```

Section C — Find and fix the bug

C1. The player's health never actually changes, and Python reports no error at all. Why not, and what is [4 marks]
the fix?

```
player_health = 20

def take_damage(amount):
    player_health = player_health - amount
```

Careful: this one *does* raise an error, and it is not the one you might expect. Explain exactly what Python does here.

C2. Moving to the tunnel crashes with `KeyError: 'tunnel'`. What is wrong?

[3 marks]

```
ROOMS = {
    "cavern": {"exits": {"north": "tunnel"}},
    "tunnel": {"exits": {"south": "cavern"}},
}
```

C3. This function is supposed to only display things, but it has a bug that will cause confusing problems [4 marks]
later. Find it, and say which rule it breaks.

```
def show_status():
    global turns
    turns = turns + 1
    print("health:", player_health)
    print("turn:", turns)
```

Section D — Build it

Build your own text adventure. At least four rooms, and make it *yours* — a spaceship, a school at night, a sunken ship.

- ☐ **Checkpoint 1.** A `while` loop that prints something, reads a command, and quits on `quit`. Check that it really stops.
- ☐ **Checkpoint 2.** Add state: health and an inventory. Write `show_status()` that displays them and **changes nothing**.
- ☐ **Checkpoint 3.** Build a `ROOMS` dictionary with at least four rooms and working exits. Write `go(direction)` once, so it works for every room.

- ☐ **Checkpoint 4.** Put items in rooms. Add `take`, so an item moves from the room's list into the inventory. Print the room's items when you arrive.
- ☐ **Checkpoint 5.** Add a way to lose (health reaching 0) and a way to win (reaching a room, or carrying a particular item). Make sure both end the loop properly.
- ☐ **Checkpoint 6.** Add a locked door. Put the requirement in the room *data*, not as a special case inside `go()` — so a second locked door needs no new code.

IF SOMETHING CHANGES AND THEN UN-CHANGES

You are missing a `global`. Python has quietly created a *new* local variable

inside your function, changed that, and thrown it away when the function ended. No error is raised. This is the most confusing thing in Python for beginners, and everybody hits it once.

Section E — Design challenge

NO RIGHT ANSWER

E1. Your rooms are dictionaries. Design the data for something more interesting: a room that is dark until you have a torch; a monster that only appears sometimes; a room that changes after you visit it. Write what one of your room dictionaries would look like.

E2. You now have about five global variables. If you added magic, money, time of day and a quest log, how many would there be? Is that a problem? What would you do instead?

E3. The one that matters. Your game waits forever at `input()`. Suppose you wanted a torch that burns out after two minutes of **real time**, whether or not the player types anything. What breaks? Sketch what would have to change.

Stretch goals

1. `take` and `drop`. Decide where you store which items are where, and say why.
2. A locked door whose requirement lives in the data.

3. **Save and load** with `json.dump` and `json.load`. If your state really is all in those variables, this is about four lines. If it is not, you will find out immediately.
4. A monster that moves one room each turn — your first taste of something happening without the player doing it.
5. Ten rooms. Draw the map on paper first. Notice you changed no code.